

MENG25301 — Microcontrollers Cheat Sheet

TM4C123GH6PM (ARM Cortex-M4) | GPIO · UART · I2C · SPI · Timers · Interrupts · ADC · PWM

► GPIO — General Purpose I/O

Clock & Pin Setup

SYSTCL->RCGCGPIO |= (1<<5); //

A=0,B=1,C=2,D=3,E=4,F=5

while(!(SYSTCL->PRGPIO & (1<<5))); // wait ready

GPIOF->DIR |= 0x0E; // PF1/2/3 OUTPUT (1=out,0=in)

GPIOF->DIR &= ~0x11; // PF0/4 INPUT

GPIOF->DEN |= 0x1F; // digital enable (required)

GPIOF->PUR |= 0x11; // pull-up (buttons active-LOW)

GPIOF->PDR |= 0x04; // pull-down

GPIOF->DATA ^= 0x04; // toggle PF2 (Blue LED)

Alternate Function (e.g. UART PA0/PA1)

GPIOA->AFSEL |= 0x03;

GPIOA->PCTL=(GPIOA->PCTL&~0xFF)|0x11; // UART=1

GPIOA->DEN |= 0x03;

■ PF0 LOCKED: write 0x4C4F434B->LOCK, set CR bit0

► UART — Asynchronous Serial

Frame: [START | 5-8 data | PARITY? | 1-2 STOP]

Baud Rate (SysClk=16MHz, Baud=9600)

ClkDiv = SysClk / (16 x Baud) = 16M/16/9600 = 104.167

IBRD = 104 (integer part)

FBRD = round(0.167 x 64 + 0.5) = 11

Reg	Purpose	Key bits
UARTIBRD	Integer baud divisor	—
UARTFBRD	Fractional baud div	—
UARTLCRH	Line control	WLEN[6:5],FE N[4],STP2[3]
UARTCTL	Control	UARTEN[0],H SE[5],RXE[8], TXE[9]
UARTDR	Data R/W	bits[7:0]
UARTFR	Flags	TXFF[5],RXF E[4],BUSY[3]

WLEN: 00=5b 01=6b 10=7b 11=8b | LCRH=0x60→8N1

Init (UART0, PA0/PA1, 9600 8N1)

SYSTCL->RCGCUART|=1; SYSTCL->RCGCGPIO|=1;

GPIOA->DEN=0x03;GPIOA->AFSEL=0x03;GPIOA->PCTL=0x11;

UART0->CTL =0; // 1. disable

UART0->IBRD=104;UART0->FBRD=11; // 2. baud

UART0->LCRH=0x60; // 3. 8N1 no FIFO

UART0->CTL =0x301; // 4. en+TXE+RXE

► SPI (SSI) — 4-Wire Synchronous Full-Duplex

Wires: MOSI MISO SCLK SS(CS) Master drives clock

Mode	CPOL	CPHA	Sample Edge
0	0	0	Rising / Change:Falling
1	0	1	Falling / Change:Rising
2	1	0	Falling / Change:Rising
3	1	1	Rising / Change:Falling

Module	Pins
SSI0	PA2-PA5
SSI1	PF0-PF3 / PD0-PD3
SSI2	PB4-PB7
SSI3	PD0-PD3

Config Steps

1. SYSTCL->RCGCSSI|=(1<<n); // clock SSIn

2. SSI0->CR1 &= ~2; // disable

3. Set CPSR prescaler & CR0 (mode/size/rate)

4. SSI0->CR1 &= ~4; // master (bit2=0)

5. SSI0->CR1 |= 2; // enable

6. Assert SS; wait TNF(SR bit1); write SSI0->DR

7. Wait TFE & !BSY; deassert SS

► I2C — 2-Wire Half-Duplex Multi-Master

Lines: SDA + SCL open-drain, 4.7kΩ pull-ups

Protocol: START→[7b ADDR][R/W]→ACK→[8b DATA]→ACK→STOP

START: SDA low while SCL high

STOP: SDA high while SCL high

ACK: receiver pulls SDA low on 9th clock

Up to 128 devices (7-bit addressing)

Reg	Purpose
I2CMSA	Master Slave Addr — addr[7:1]+R/W[0]
I2CMCS	Control/Status — RUN,START,STOP,ACK
I2CMDR	Master Data Register (TX/RX byte)

Module	SCL	SDA
I2C0	PB2	PB3
I2C1	PA6	PA7
I2C2	PE4	PE5
I2C3	PD0	PD1

■ Init: RCGCI2C, GPIO AFSEL/PCTL/DEN/ODR, I2CMCR=0x10

► Timers — SysTick (24-bit down-counter)

Reg	Purpose	Key Bits
CTRL	Control	[0]=EN [1]=INTEN [2]=CLK_SRC
LOAD	Reload	[23:0] max=0xFFFFF=16,777, 215
VAL	Current	Write any → clear to 0

Formulas

Delay = (N+1)/sysclk => N = Delay*sysclk - 1

1ms@16MHz: N=15999 | Max@16MHz: ~1.049s

For >1s: loop multiple 1-second countdowns

Polling Pattern

SysTick->CTRL=0; // disable

SysTick->LOAD=0x05; // EN+sysclk, no IRQ

void delay_ms(uint32_t ms){

for(uint32_t i=0;i<ms;++){

SysTick->LOAD=15999; SysTick->VAL=0;

while(!(SysTick->CTRL&0x10000));}}

Free-Run + Overflow ISR (elapsed time)

SysTick->LOAD=0x0FFFFFFF; SysTick->VAL=0;

SysTick->CTRL=0x07; // EN+INTEN+sysclk

void SysTick_Handler(void){overflows++;

// ticks=dOverflows*(0xFFFFF+1)+(t1-t2)

// time_us = ticks/16 (16ticks/us@16MHz)

GPTM: 12 blocks, supports

PWM/capture/count/compare/IRQ

► Interrupts & NVIC

3-Step Enable (example: PORTF)

// 1. Peripheral level

GPIOF->IM |= 0x11; // unmask PF0 & PF4

// 2. NVIC level

NVIC->ISER[0]|=(1<<30); // IRQ30=PORTF

// 3. Global

__enable_irq(); // __disable_irq() to mask

GPIO Trigger Registers

GPIOF->IS &=~0x11; // 0=edge, 1=level

GPIOF->IBE &=~0x11; // 0=single edge

GPIOF->IEV &=~0x11; // 0=falling, 1=rising

IS	IEV	Trigger
0	0	Falling edge
0	1	Rising edge
1	0	Low level
1	1	High level

ISR Template

void GPIOF_Handler(void){

if(GPIOF->MIS&0x10){/* PF4/SW1 */}

if(GPIOF->MIS&0x01){/* PF0/SW2 */}

GPIOF->ICR|=0x11; // MUST clear flag!

}

Priority: NVIC->IP[n] = prio<=5 (0=high, 7=low)

IRQ	Device	IRQ	Device
0	GPIO Port A	4	GPIO Port E
1	GPIO Port B	5	UART0
2	GPIO Port C	8	I2C0
3	GPIO Port D	10	PWM0 Gen0
14	ADC0 Seq0	19	Timer 0A
17	ADC0 Seq3	30	GPIO Port F

Auto-pushed on IRQ: R0-R3, R12, LR, PC, xPSR

Flow: IRQ→NVIC→auto-push→vector lookup→ISR→BX LR

► ADC — Analog to Digital Converter

Formulas

Levels = 2^n

Resolution = Vref/2^n (V/LSB)

Code = Vin*(2^n-1)/Vref

Vin = (Vref*K)/(2^n-1) [K=ADC value]

Nyquist: Fs >= 2*Fmax

TM4C: 12-bit, Vref=3.3V, step=0.806mV

LM34: Temp(F)=SSFIFO3*330/4096

Resolution (Vref=5V)

8-bit:19.53mV 10-bit:4.88mV 12-bit:1.22mV

AIN	Pin	AIN	Pin
AIN0	PE3	AIN6	PD1
AIN1	PE2	AIN7	PD0
AIN2	PE1	AIN8	PE5
AIN3	PE0	AIN9	PE4
AIN4	PD3	AIN10	PB4
AIN5	PD2	AIN11	PB5

SS3 Init (single sample, software trigger)

SYSTCL->RCGCADC|=1; SYSTCL->RCGCGPIO|=

=(1<<4);

GPIOE->AFSEL|=0x08; GPIOE->DEN&=~0x08;

GPIOE->AMSEL|=0x08; // analog enable

ADC0->ACTSS &=~8; // disable SS3

ADC0->EMUX &=~0xF000; // SW trigger

ADC0->SSMUX3 =0; // AIN0

ADC0->SSCTL3 =6; // IE0+END0

ADC0->ACTSS |=8; // enable SS3

Read Sample

ADC0->PSSI|=8; // start conversion

while(!(ADC0->RIS&8)); // wait INT3

result=ADC0->SSFIFO3&0xFF; // 12-bit result

ADC0->ISC=8; // clear flag

MENG25301 — Microcontrollers Cheat Sheet

TM4C123GH6PM (ARM Cortex-M4) | GPIO · UART · I2C · SPI · Timers · Interrupts · ADC · PWM

► PWM — Pulse Width Modulation

Formulas (Count-Down Mode)

DC% = TON/TPWM * 100%

LOAD = PWM_clk / freq

CMPA = (1 - DC%) * LOAD

e.g. 1kHz 25%@16MHz: LOAD=16000,CMPA=12000

e.g. 5kHz 60%@16MHz: LOAD=3200, CMPA=1280

Reg	Purpose
PWMxLOAD	Period (sets frequency)
PWMxCMPA/B	Compare (duty cycle)
PWMxCCTL	Start/stop; bit1=up-down mode
PWMxGENA/B	Output action on events
PWM0/1->ENABLE	Route to output pin

PWM0 & PWM1 | 4 generators each | 8 outputs each

Example: M1PWM7 on PF3 — 1kHz 50%DC

SYSTCL->RCGCPWM|=2; // clock PWM1

SYSTCL->RCGCGPIO|=1<<5); // clock Port F

SYSTCL->RCC &=~(1<<20); // clear USEPWMDIV

GPIOF->AFSEL|=0x08;

GPIOF->PCTL=(GPIOF->PCTL&~0xF000)|0x5000; // M1PWM7=5

GPIOF->DEN|=0x08;

PWM1->_3_CTL =0; // disable gen3

PWM1->_3_GENB=0x0C8; // HI@load, LO@CMPB

PWM1->_3_LOAD=16000; // 16MHz/1kHz

PWM1->_3_CMPB=8000; // (1-0.5)*16000

PWM1->_3_CTL =1; // enable gen3

PWM1->ENABLE|=(1<<7); // enable M1PWM7

■ PF3 PCTL: GPIO=0,UART=1,SSI=2,PWM(M1PWM7)=5

► Quick Reference

Port F Pin Map

Pin	Mask	Device	Dir
PF0	0x01	SW2 (LOCKED)	In pull-up
PF1	0x02	Red LED	Output
PF2	0x04	Blue LED	Output
PF3	0x08	Green LED/PWM	Output
PF4	0x10	SW1	In pull-up

Minimal LED Blink

SYSTCL->RCGCGPIO|=(1<<5);

while(!(SYSTCL->PRGPIO&(1<<5)));

GPIOF->DIR|=0x04; GPIOF->DEN|=0x04;

while(1){GPIOF->DATA^=0x04;delay_ms(500);}

Read Active-LOW Button

if(!(GPIOF->DATA&0x10)){/* SW1 pressed */}

Parity Bit Calculation

Even: total 1s (data+parity) = EVEN

Odd: total 1s (data+parity) = ODD

10111010 has 5 ones → even parity=1 (total 6)

10110010 has 4 ones → odd parity=1 (total 5)

SysTick CTRL Bits

Bit	Name	Effect
0	ENABLE	Enable SysTick
1	INTEN	IRQ on underflow
2	CLK_SRC	1=sysclk, 0=int osc
16	COUNT	Set on underflow (r/o)

0x05=poll mode (EN+CLK) 0x07=IRQ mode (+INTEN)

ADC Code Examples (Vin=3V, Vref=5V)

8-bit: 3*255/5 = 153 (0x99)

10-bit: 3*1023/5 = 614 (0x266)

12-bit: 3*4095/5 = 2457 (0x999)

UART Flag Register (UARTFR)

Bit	Name	Meaning
3	BUSY	UART transmitting
4	RXFE	RX FIFO empty
5	TXFF	TX FIFO full
6	RXFF	RX FIFO full
7	TXFE	TX FIFO empty

Serial Interface Comparison

Proto	Wires	Duplex	Sync	Speed
UART	2(TX,RX)	Full	Async	~115kbp s
SPI	4	Full	Sync	~10Mbps
I2C	2(SDA,SCL)	Half	Sync	~400kbp s

► Interrupt Vector Table (ARM Cortex-M)

INT#	Interrupt	Addr
1	Reset (startup)	0x00000004
2	NMI	0x00000008
3	Hard Fault	0x0000000C
11	SVCall	0x0000002C

INT#	Interrupt	Addr
14	PendSV	0x00000038
15	SysTick	0x0000003C
16+	IRQ0+ (peripherals)	0x00000040+

First 1024 bytes reserved for NVIC (256 x 4 bytes)
Priority: Reset=-3, NMI=-2, HardFault=-1, rest=0-7
ARM Cortex-M Processor Modes

Mode	Triggered by	Privilege
Thread	Reset / return from ISR	Priv or Unpriv
Handler	Any exception / IRQ	Always Privileged

Stack pointers: MSP (main) used in Handler mode; PSP (process) in Thread mode
► Nested Interrupt Handling
Higher-priority IRQ can interrupt a running ISR
NVIC manages preemption automatically
Interrupt latency = time from IRQ to ISR start
Context save/restore is automatic (hardware)
► GPIO Interrupt Registers Summary

Reg	Purpose
GPIOIM	Interrupt Mask — enable/disable per pin
GPIOIS	Sense — 0=edge, 1=level
GPIOIBE	Both Edges — 1=both edges trigger
GPIOIEV	Event — 0=falling/low, 1=rising/high
GIORIS	Raw Interrupt Status
GIOMIS	Masked Interrupt Status (use in ISR)
GPIOICR	Interrupt Clear — write 1 to clear

► Key Constants & Values
16 MHz sysclk → 1 tick = 62.5 ns
16 MHz sysclk → 16 ticks per microsecond
SysTick max LOAD = 0xFFFFF = 16,777,215
SysTick max delay@16MHz ≈ 1.049 seconds
TM4C ADC: 12-bit, 0-4095, Vref=3.3V
ADC step size = 3.3V/4096 = 0.806 mV/LSB
UART 9600@16MHz: IBRD=104, FBRD=11
PWM PCTL M1PWM7 on PF3 = value 5
IRQ30 = GPIO Port F
NVIC ISER[0] bit30 → enables IRQ30 (Port F)
Common #include / Peripheral Base Addresses
#include "TM4C123GH6PM.h"
GPIOF base: 0x40025000
UART0 base: 0x4000C000
ADC0 base: 0x40038000
PWM1 base: 0x40029000
SysTick: 0xE000E010 (CorePeripherals)

Configure Interrupt
void GPIOF_Init_Interrupt(void){
 SYSCCTL->RCGCGPIO |= (1<<5);
 while(!(SYSCCTL->PRGPIO & (1<<5)));

 GPIOF->DIR &= ~0x10; // PF4 input
 GPIOF->DEN |= 0x10;
 GPIOF->PUR |= 0x10;

 GPIOF->IS &= ~0x10; // edge
 GPIOF->IBE &= ~0x10; // single edge
 GPIOF->IEV &= ~0x10; // falling edge
 GPIOF->ICR |= 0x10; // clear flag
 GPIOF->IM |= 0x10; // unmask

 NVIC->ISER[0] |= (1<<30); // enable IRQ30
 __enable_irq();
}
Timer (SysTick)
void SysTick_Init(void){
 SysTick->CTRL = 0;
 SysTick->LOAD = 0x0FFFFFFF;
 SysTick->VAL = 0;
 SysTick->CTRL = 0x05; // enable, no interrupt
}

void delay_us(uint32_t us){
 for(uint32_t i=0;i<us;i++){
 SysTick->LOAD = 16-1; // 1us @16MHz
 SysTick->VAL = 0;
 while(!(SysTick->CTRL & 0x10000));
 }
}
► This gives microsecond + millisecond flexibility
Send String
void UART0_SendString(char *str){
 while(*str){
 while(UART0->FR & 0x20); // wait if TX full
 UART0->DR = *str;
 str++;
 }
}
SPI Transfer Function
uint8_t SPI_Transfer(uint8_t data){
 while(!(SSI0->SR & 0x02)); // TNF
 SSI0->DR = data;

 while(SSI0->SR & 0x10); // wait BSY
 return SSI0->DR;
}
► write then read from SPI device; full duplex
explanation
I2C Write + Read Pattern
void I2C_Write(uint8_t addr, uint8_t data){
 I2C0->MSA = (addr<<1); // write
 I2C0->MDR = data;
 I2C0->MCS = 0x07; // START + RUN + STOP
 while(I2C0->MCS & 0x01); // wait busy
}

uint8_t I2C_Read(uint8_t addr){
 I2C0->MSA = (addr<<1) | 1; // read
 I2C0->MCS = 0x07;
 while(I2C0->MCS & 0x01);
 return I2C0->MDR;
}
► Write to sensor; Read from register
ADC → Voltage Conversion
float ADC_to_Voltage(uint16_t adc){
 return (adc * 3.3) / 4095.0;
}
► Converts theory → practical answer
ADC → Temperature
float TempF(uint16_t adc){
 return (adc * 330.0) / 4096.0;
}
PWM Duty Cycle Function
void PWM_SetDuty(uint16_t load, float duty){
 PWM1->_3_CMPB = (1.0 - duty) * load;
}
► PWM_SetDuty(16000, 0.25); // 25%
Main Loop Pattern
int main(void){
 GPIO_Init();
 UART0_Init();
 SysTick_Init();

 while(1){
 if(!(GPIOF->DATA & 0x10)){
 UART0_SendString("Pressed\n");
 delay_us(500000);
 }
 }
}

► write a full program; integration questions
Button Interrupt Trigger
GPIOF->IS &= ~0x10;
GPIOF->IEV &= ~0x10; // falling
GPIOF->IM |= 0x10;
NVIC->ISER[0] |= (1<<30);
UART Receive Char
while(UART0->FR & 0x10);
char c = UART0->DR;
ADC Read (SS3)
ADC0->PSSI |= 8;
while(!(ADC0->RIS & 8));
val = ADC0->SSFIFO3 & 0xFFFF;
ADC0->ISC = 8;
INITIALIZATION ORDER
1. Enable clock (SYSCCTL->RCGCxxx)
2. Wait ready (PRxxx)
3. Configure GPIO (DIR, DEN, AFSEL, PCTL)
4. Configure peripheral registers
5. Enable peripheral
6. Enable NVIC (if interrupt)
Bitwise Operations
SET BIT: reg |= (1<<n);
CLEAR BIT: reg &= ~(1<<n);
TOGGLE: reg ^= (1<<n);
CHECK: if(reg & (1<<n))
reg &= ~0x0F; // clear lower 4 bits
reg |= 0x05; // set bits 0 and 2
Polling vs Interrupt (MCQ gold)
POLLING:
- CPU waits (while loop)
- simple but wastes CPU

INTERRUPT:
- event-driven
- CPU free until trigger
- requires NVIC + ISR
Edge vs Level Trigger
Edge → one-time event (button press)
Level → continuous while condition true

Falling edge → button press (active LOW)
UART / SPI / I2C — WHEN to use each
UART → simple, 2 devices, long distance
SPI → fast, short distance, many slaves
I2C → multiple devices, fewer wires
Throughput / Timing Insight
UART baud = bits/sec
SPI speed = clock rate
I2C speed limited by pull-ups

Higher clock → faster but less reliable
ADC Sampling Concept

Higher sampling rate → better signal accuracy
Aliasing if Fs < 2*Fmax
Interrupt Flow
Event → Flag set → NVIC → ISR → clear flag

- ALWAYS enable clock before config
- ALWAYS clear interrupt flag in ISR
- ALWAYS enable DEN for GPIO
- ALWAYS wait for peripheral ready

► **INITIALIZATION ORDER (ALWAYS)**
1. Enable clock (SYSCTL->RCGCxxx)
2. Wait ready (PRxxx)
3. Configure GPIO (DIR, DEN, AFSEL, PCTL)
4. Configure peripheral registers
5. Enable peripheral
6. Enable NVIC (if interrupt)

► **BITWISE OPERATIONS**
SET: reg |= (1<<n);
CLEAR: reg &= ~(1<<n);
TOGGLE: reg ^= (1<<n);
CHECK: if(reg & (1<<n))

MASKING:
reg &= ~0x0F; // clear bits
reg |= 0x05; // set bits

► **GPIO**
SYSCTL->RCGCGPIO |= (1<<5);
while(!(SYSCTL->PRGPIO & (1<<5)));

GPIOF->DIR |= 0x0E;
GPIOF->DIR &= ~0x11;
GPIOF->DEN |= 0x1F;
GPIOF->PUR |= 0x11;

GPIOF->DATA ^= 0x04;

ALT FUNC (UART):
GPIOA->AFSEL |= 0x03;
GPIOA->PCTL = (GPIOA->PCTL & ~0xFF) | 0x11;
GPIOA->DEN |= 0x03;

■ **PF0 LOCK:** write 0x4C4F434B → LOCK, set CR bit0

► **UART**
IBRD=104, FBRD=11 (9600 @16MHz)

INIT:
SYSCTL->RCGCUART |=1;
SYSCTL->RCGCGPIO |=1;
GPIOA->DEN=0x03; GPIOA->AFSEL=0x03; GPIOA->PCTL=0x11;

UART0->CTL=0;
UART0->IBRD=104; UART0->FBRD=11;
UART0->LCRH=0x60;
UART0->CTL=0x301;

SEND STRING:
while(*s){
while(UART0->FR & 0x20);
UART0->DR = *s++;
}
}

RECEIVE:
while(UART0->FR & 0x10);
char c = UART0->DR;

► **SPI (SSI)**
while(!(SSI0->SR & 0x02));
SSI0->DR = data;
while(SSI0->SR & 0x10);
return SSI0->DR;

► **I2C**
WRITE:
I2C0->MSA = (addr<<1);
I2C0->MDR = data;
I2C0->MCS = 0x07;
while(I2C0->MCS & 1);

READ:
I2C0->MSA = (addr<<1)|1;
I2C0->MCS = 0x07;
while(I2C0->MCS & 1);
data = I2C0->MDR;

► **SYSTICK TIMER**
Delay = (N+1)/sysclk

INIT:
SysTick->CTRL=0;
SysTick->LOAD=0x0FFFFFFF;
SysTick->VAL=0;
SysTick->CTRL=0x05;

DELAY (us):
SysTick->LOAD = 16-1;
SysTick->VAL=0;
while(!(SysTick->CTRL & 0x10000));

► **INTERRUPTS**
ENABLE:
GPIOF->IM |= 0x10;

NVIC->ISER[0] |= (1<<30);
__enable_irq();

TRIGGER:
GPIOF->IS &= ~0x10;
GPIOF->IEV &= ~0x10;

ISR:
void GPIOF_Handler(void){
if(GPIOF->MIS & 0x10){
GPIOF->ICR |= 0x10;
}
}

FLOW:
Event → Flag → NVIC → ISR → clear flag

► **ADC**
FORMULAS:
Code = Vin*(2^n-1)/Vref
Vin = (Vref*K)/(2^n-1)

INIT:
SYSCTL->RCGCADC|=1;
SYSCTL->RCGCGPIO|=(1<<4);

GPIOE->AFSEL|=0x08;
GPIOE->DEN&=~0x08;
GPIOE->AMSEL|=0x08;

ADC0->ACTSS &=~8;
ADC0->EMUX &=~0xF000;
ADC0->SSMUX3 =0;
ADC0->SSCTL3 =6;
ADC0->ACTSS |=8;

READ:
ADC0->PSSI|=8;
while(!(ADC0->RIS&8));
val = ADC0->SSFIFO3 & 0xFFFF;
ADC0->ISC=8;

VOLTAGE:
V = (adc * 3.3) / 4095.0;

TEMP:
TempF = (adc * 330.0) / 4096.0;

► **PWM**
LOAD = PWM_clk / freq
CMP = (1 - duty) * LOAD

SET DUTY:
PWM1->_3_CMPB = (1.0 - duty) * load;

► **QUICK CODE PATTERNS**
LED BLINK:
GPIOF->DIR |= 0x04;
GPIOF->DEN |= 0x04;
while(1){GPIOF->DATA ^= 0x04;}

BUTTON:
if(!(GPIOF->DATA & 0x10)){}

► **CONCEPTS (MCQ GOLD)**

POLLING:
- CPU waits

INTERRUPT:
- event-driven

EDGE vs LEVEL:
- Edge = one event
- Level = continuous
- Falling edge = button press

COMMUNICATION:
UART → simple, long distance
SPI → fast
I2C → multiple devices

ADC:
- $F_s \geq 2F_{\max}$ (Nyquist)
- aliasing if violated

TIMING:
UART baud = bits/sec
SPI speed = clock
I2C limited by pull-ups

► **MUST REMEMBER**
- Enable clock FIRST
- Wait for ready
- Enable DEN for GPIO
- Clear interrupt flags

Q: Why use interrupt instead of polling?
A: Saves CPU, event-driven

Q: Why pull-up resistor?
A: Prevent floating input

Q: Why Nyquist?
A: Avoid aliasing

Q: Why SPI faster than I2C?
A: No addressing overhead + push-pull

► **GPIO BITMASK PATTERNS**

ENABLE PORT CLOCK:
A=0x01 B=0x02 C=0x04 D=0x08 E=0x10 F=0x20
Example:
SYSCTL->RCGCGPIO |= 0x03; // A+B
SYSCTL->RCGCGPIO |= 0x31; // A+E+F

SET PINS AS OUTPUT:
GPIOx->DIR |= mask;
GPIOx->DEN |= mask;

Examples:
PA0 → 0x01
PA0,3,6 → 0x49
PB2,3,4 → 0x1C

SET INPUT + PULL-UP:
GPIOx->DIR &= ~mask;
GPIOx->DEN |= mask;
GPIOx->PUR |= mask;

Example:
PD2,3,4 → mask=0x1C

WRITE TO PINS:

TURN ON:
GPIOx->DATA |= mask;

TURN OFF:
GPIOx->DATA &= ~mask;

TOGGLE:
GPIOx->DATA ^= mask;

IMPORTANT:
"WITHOUT AFFECTING OTHER PINS"
→ ALWAYS use |= or &= ~
→ NEVER use =

► **QUICK MASK BUILDING**

Bit n = 1<<n

Examples:
PA3 → (1<<3) = 0x08
PB2,3,4 → (1<<2)|(1<<3)|(1<<4) = 0x1C

MENG25301 — Microcontrollers Cheat Sheet TM4C123GH6PM (ARM Cortex-M4) GPIO · UART · I2C · SPI · Timers · Interrupts · ADC · PWM ► INITIALIZATION ORDER (ALWAYS) 1. Enable clock (SYSCTL->RCGCxxx) 2. Wait ready (PRxxx) 3. Configure GPIO (DIR, DEN, AFSEL, PCTL) 4. Configure peripheral 5. Enable peripheral 6. Enable NVIC (if interrupt) ► BITWISE OPERATIONS SET: reg = (1<<n); CLEAR: reg &= ~(1<<n); TOGGLE: reg ^= (1<<n); CHECK: if(reg & (1<<n)) MASKING: reg &= ~0x0F; reg = 0x05; ► GPIO (CORE PATTERN) SYSCTL->RCGCGPIO = (1<<5); while(!(SYSCTL->PRGPIO & (1<<5))); GPIOF->DIR = 0x0E; GPIOF->DIR &= ~0x11; GPIOF->DEN = 0x1F; GPIOF->PUR = 0x11; GPIOF->DATA ^= 0x04; ALT FUNC (UART): GPIOA->AFSEL = 0x03; GPIOA->PCTL = (GPIOA->PCTL & ~0xFF) 0x11; GPIOA->DEN = 0x03; ■ PF0 LOCK → 0x4C4F434B ► UART (9600 @16MHz) IBRD=104, FBRD=11 INIT: SYSCTL->RCGCUART =1; SYSCTL->RCGCGPIO =1; GPIOA->DEN=0x03; GPIOA->AFSEL=0x03; GPIOA->PCTL=0x11; UART0->CTL=0; UART0->IBRD=104; UART0->FBRD=11; UART0->LCRH=0x60; UART0->CTL=0x301; SEND: <pre>while(*s){ while(UART0->FR & 0x20); UART0->DR = *s++; }</pre> RECEIVE: <pre>while(UART0->FR & 0x10); char c = UART0->DR;</pre> ► SPI <pre>while(!(SSI0->SR & 0x02)); SSI0->DR = data; while(SSI0->SR & 0x10); return SSI0->DR;</pre> ► I2C WRITE: <pre>I2C0->MSA = (addr<<1); I2C0->MDR = data; I2C0->MCS = 0x07; while(I2C0->MCS & 1);</pre> READ: <pre>I2C0->MSA = (addr<<1) 1; I2C0->MCS = 0x07; while(I2C0->MCS & 1); data = I2C0->MDR;</pre> ► SYSTICK INIT: <pre>SysTick->CTRL=0; SysTick->LOAD=0x00FFFFFF; SysTick->VAL=0; SysTick->CTRL=0x05;</pre> DELAY (1us): <pre>SysTick->LOAD = 16-1; SysTick->VAL=0; while(!(SysTick->CTRL & 0x10000));</pre> ► INTERRUPTS ENABLE: <pre>GPIOF->IM = 0x10; NVIC->ISER[0] = (1<<30);</pre>	__enable_irq(); ISR: <pre>void GPIOF_Handler(void){ if(GPIOF->MIS & 0x10){ GPIOF->ICR = 0x10; } }</pre> FLOW: Event → NVIC → ISR → CLEAR FLAG ► ADC Code = $Vin \cdot (2^n - 1) / Vref$ $Vin = (Vref \cdot K) / (2^n - 1)$ READ: <pre>ADC0->PSSI =8; while(!(ADC0->RIS&8)); val = ADC0->SSFIFO3 & 0xFFFF; ADC0->ISC=8;</pre> Voltage: $V = (adc * 3.3) / 4095.0;$ Temp: $TempF = (adc * 330.0) / 4096.0;$ ► PWM LOAD = PWM_clk / freq CMP = (1 - duty) * LOAD $PWM1 \rightarrow _3_CMPB = (1.0 - duty) * load;$ ► QUICK PATTERNS LED: <pre>GPIOF->DIR = 0x04; GPIOF->DEN = 0x04; while(1){GPIOF->DATA ^= 0x04;}</pre> BUTTON: <pre>if(!(GPIOF->DATA & 0x10)){}</pre> ► MCQ GOLD Polling = CPU waits Interrupt = event-driven Edge = one event Level = continuous UART → simple SPI → fastest I2C → multi-device Nyquist: $F_s \geq 2F_{max}$ ► MUST REMEMBER * Enable clock FIRST * Wait ready * Enable DEN * Clear interrupt flag ► WORKSHEET PATTERNS (EXAM GOLD) PORT CLOCK MASKS: A=0x01 B=0x02 C=0x04 D=0x08 E=0x10 F=0x20 Examples: A+B → 0x03 B+D → 0x0A A+E+F → 0x31 OUTPUT PINS: <pre>GPIOx->DIR = mask; GPIOx->DEN = mask;</pre> Examples: PA0 → 0x01 PA0,3,6 → 0x49 PB2,3,4 → 0x1C INPUT + PULL-UP: <pre>GPIOx->DIR &= ~mask; GPIOx->DEN = mask; GPIOx->PUR = mask;</pre> Example: PD2,3,4 → 0x1C WRITE TO PINS: <pre>ON: GPIOx->DATA = mask; OFF: GPIOx->DATA &= ~mask; TOGGLE: GPIOx->DATA ^= mask;</pre> ■ NEVER use '=' unless resetting ALL pins MASK BUILDING: Bit n = (1<<n)
--	---

Examples:
PA3 → 0x08
PB2,3,4 → 0x1C

LED CONTROL EXAMPLE (PORT E):

GPIOE->DIR |= 0xFF;
GPIOE->DEN |= 0xFF;

ON PE0,2,5:
GPIOE->DATA |= 0x25;

SWAP PE2→OFF, PE3→ON:
GPIOE->DATA ^= 0x0C;

TURN ON PE1:
GPIOE->DATA |= 0x02;

PAGE 2 — DETAILED REFERENCE

(Keep for deeper questions, not memorization)

- * Full UART register breakdown
- * Full SPI modes table
- * I2C protocol sequence
- * SysTick math + overflow timing
- * NVIC priority + IRQ table
- * ADC resolution table + channels
- * PWM generator setup example
- * Interrupt vector table
- * Cortex-M modes + stack behavior

(Use only if exam asks THEORY or LONG FORM)

